



Esonero 2 — Arcade Racing Game

Brief, deliverable e criteri di valutazione

10 maggio 2026

Docente: Ennio Pirolo
Corso: GVD1 — Tecniche dei Nuovi Media Integrati
Anno 2025/2026
Accademico:
Consegna: Giovedì 4 giugno 2026

Premessa

Il primo esonero era partito da una traccia precisa — un hidden object game — ma molti di voi sono andati ben oltre: c'è chi ha costruito un'esperienza in prima persona, chi ha intrecciato meccaniche di esplorazione, chi ha aggiunto sistemi che non erano nella richiesta. Questa voglia di spingersi più avanti è esattamente lo spirito che voglio premiare anche in questo secondo esonero. Vi propongo quindi un cambio di passo: passiamo dalla logica statica di un livello da scrutare al *game feel* di un arcade in tempo reale. L'obiettivo non è imparare a programmare un veicolo da zero — quello richiederebbe un corso a parte — ma imparare a **progettare un gioco** partendo da un toolkit di pezzi pronti, e a comporli in modo originale.

Vi consegno un'auto che si muove, una pista vuota e una libreria di mattoncini configurabili. La domanda a cui dovete rispondere è: **dato tutto questo, che gioco volete farci?**

Obiettivi didattici

L'esonero verifica la vostra capacità di:

- progettare un livello arcade in 3D leggibile e divertente da percorrere;
- definire una meccanica di gioco chiara, con regole comprensibili in pochi secondi;
- comporre in modo creativo componenti pronti, configurandoli da Inspector;
- curare il *feel* di gioco attraverso audio, VFX, illuminazione e UI;
- presentare e raccontare il proprio lavoro.

Non è richiesta scrittura di codice. Tutto si fa in scena, dall'Inspector e tramite drag-and-drop di prefab.

Cosa vi fornisco io (Starter Toolkit)

All'inizio del lavoro riceverete un progetto Unity con i seguenti elementi pronti all'uso:

Sistemi core

Componente	Cosa fa	Come lo configurate
Kart Controller	Muove l'auto con fisica arcade (accelerazione, sterzo, gravità custom)	Parametri esposti nell'Inspector: velocità max, accelerazione, sensibilità sterzo, attrito
Timer Manager	Gestisce countdown o count-up del livello	Modalità (countdown/countup), tempo iniziale, cosa succede a fine tempo
Level Manager	Gestisce inizio livello, condizione di vittoria, passaggio al successivo	Lista di obiettivi richiesti, scena successiva

Libreria di mattoncini (prefab)

Prefab	Tipi di comportamento configurabili
Trigger Zone	Boost di velocità, rallentamento, teleport, danno, bonus tempo, malus tempo, attivazione di altri oggetti
Collectible	Oggetto raccogliibile con effetto a scelta (punti, tempo, sblocco, sound, VFX, respawn)
Win Condition	Raccogli N oggetti, raggiungi un punto, supera N checkpoint, sopravvivi X secondi
Lose Condition	Tempo scaduto, caduta dalla pista, danno accumulato
Moving Obstacle	Piattaforma o ostacolo che oscilla, ruota, traslata
Checkpoint	Punto di passaggio obbligato con feedback visivo e sonoro

Risorse di supporto

- Un canvas HUD pronto, con campi per timer, contatore oggetti e messaggi di stato;
- Una libreria di VFX (particellari pickup, boost, esplosioni) e SFX (motore, raccolta, vittoria);
- Una scena demo completa che mostra come si combinano i pezzi;
- Asset 3D di base per kart, pista e ambiente (potete sostituirli con altri di vostra scelta).

Cosa dovete fare voi

Deliverable richiesti

- **Un livello giocabile** end-to-end, con inizio, gioco e schermata di fine livello.
- **Una meccanica originale** che caratterizzi il vostro gioco. Non basta "raccogli i powerup nel più breve tempo possibile" — quella è la meccanica di partenza. Voglio una *vostra* variazione, una regola che lo renda interessante e diverso dagli altri.
- **Polish completo**: audio, VFX, illuminazione, UI funzionante, feedback visivo per ogni azione.
- **Un main menu** semplice con pulsante Start e Quit.

- **Un README** nel repository con nome, cognome, titolo del gioco, descrizione della meccanica in 2-3 righe e comandi base.
- **Build Windows** funzionante, oltre al progetto Unity sorgente versionato su Git.

Esempi di meccaniche originali (per ispirazione)

Per darvi un'idea del livello di originalità che cerco, ecco alcuni esempi:

- **Survival a tempo decrescente** — il timer scende, raccogliere oggetti aggiunge secondi, perdere il timer = game over;
- **Caccia al tesoro a chiavi** — la pista è divisa in segmenti chiusi, raccogli chiavi per sbloccarli;
- **Risk & reward** — esiste una scorciatoia ma è piena di hazard, vale la pena rischiare?;
- **Momentum challenge** — il kart accelera solo se mantieni una velocità minima, fermarsi è la morte;
- **Fragile run** — hai 3 vite, ogni hazard ne toglie una, il livello è una corsa al tesoro nervosa;
- **Echo run** — al secondo giro compaiono ostacoli nei punti dove sei passato al primo.

Questi sono solo spunti. La meccanica migliore è quella **vostra**, non copiata da questa lista.

Riferimenti visivi e concettuali

Quello che segue è una piccola rosa di giochi pensata per **aprire la testa** prima di mettersi a progettare. L'obiettivo non è copiarli — sono volutamente diversi tra loro, di epoche e generi differenti — ma capire che un'auto in un videogioco non è obbligata a fare delle corse: può raccontare, accompagnare, esplorare, sopravvivere, trasportare. Guardatevi almeno **un trailer o gameplay** di ognuno prima di scegliere la vostra meccanica.

Wheels of Aurelia — Santa Ragione, 2016

Top-down narrativo lungo la costiera tirrenica negli anni '70. Guidi Lella, una donna inquieta, e la "vera" meccanica del gioco è il **dialogo** con i passeggeri che raccoglie. Sedici finali diversi a seconda delle scelte. L'auto è il pretesto, la conversazione è il gioco. Studio italiano, ulteriore motivo per guardarlo.

Cosa insegna: il core loop di un gioco può essere una conversazione, non una meccanica d'azione. Anche un genere "vecchio" come l'arcade isometrico può ospitare un gioco totalmente nuovo se cambi cosa chiedi al giocatore di fare.

- Sito ufficiale: wheelsofaurelia.com
- Steam (screenshot ufficiali): store.steampowered.com/app/508740
- Gameplay completo no-commentary: youtube.com/watch?v=h8llegsWk40

Art of Rally — Funselektor, 2020

Top-down isometrico, palette satura, modelli low-poly, l'auto piccolissima in paesaggi enormi. Atmosfere lo-fi e quasi meditative. Il gioco è una corsa rally, ma l'identità visiva è così forte e coerente che funziona come un quadro animato.

Cosa insegna: uno **stile visivo unitario** vale più di mille texture realistiche. Anche con poche forme, se la palette e la luce sono coerenti, il gioco "esiste" come oggetto estetico riconoscibile.

- Sito ufficiale: artofrally.com
- Steam (screenshot ufficiali): store.steampowered.com/app/550320

- Gameplay walkthrough no-commentary: youtube.com/watch?v=ZhFUwNmKp5w

Lonely Mountains: Downhill — Megagon Industries, 2019

Non è un'auto, è una bici, ma lo metto comunque perché è il manifesto del *physics-driven minimalism*: poligoni piatti, palette decise, una sensazione di velocità e tensione date solo da composizione, camera e sound design.

Cosa insegna: con **pochissimi mezzi** si può ottenere un fortissimo *feel*. Non serve fotorealismo per sentire la discesa nelle gambe.

- Sito ufficiale: lonelymountains.com
- Steam (screenshot ufficiali): store.steampowered.com/app/711540
- Gameplay primi 15 minuti no-commentary: youtube.com/watch?v=xwx7dyaTAP0

Slow Roads — Anslo, 2022 (browser)

Guida infinita in terza persona, paesaggio generato proceduralmente, niente gare, niente timer, niente vittoria. Solo strada, montagne, mare, alba e tramonto. Si gioca direttamente nel browser.

Cosa insegna: un gioco può chiedervi di **rallentare**, non di battere un record. Togliere obiettivi è una scelta di design legittima, e a volte produce esperienze più memorabili di mille livelli a punteggio.

- Gioca direttamente nel browser: slowroads.io

Jalopy — MinskWorks, 2018

Viaggio in solitaria attraverso l'Europa post-sovietica a bordo di una Laika 601 scassata. Il "core loop" non è la velocità ma la **cura**: cambia gomme, controlla l'olio, gestisci i bagagli, fai benzina. L'auto non è un mezzo ma un **compagno di viaggio** da mantenere in vita. Visivamente low-poly, palette desaturata, ritmo lento.

Cosa insegna: l'auto può essere un **personaggio**, non solo uno strumento. Spostare il focus dal "vincere" al "prendersi cura" cambia completamente l'esperienza.

- Steam (screenshot ufficiali): store.steampowered.com/app/446020
- Wikipedia: [en.wikipedia.org/wiki/Jalopy_\(video_game\)](https://en.wikipedia.org/wiki/Jalopy_(video_game))
- Gameplay no-commentary: youtube.com/watch?v=tbiY5_ewrXo

Super Off Road — Tradewest, 1989 arcade / 1991 SNES

Il classico. Vista dall'alto fissa, mini circuiti sterrati a forma di otto/ovale, tre piloti contemporaneamente in pista, e a fine gara compri **upgrade** alla macchina con i soldi del piazzamento (gomme, sospensioni, motore, nitros). Loop arcade puro con meta-progressione tra i livelli.

Cosa insegna: la **meta-progressione tra livelli** — guadagni qualcosa nel livello 1 che migliora il livello 2 — è uno dei pattern più potenti del game design. Mappabile direttamente sul nostro toolkit (esempio: monete raccolte boost partenza nel livello successivo).

- Wikipedia: en.wikipedia.org/wiki/Super_Off_Road
- Galleria screenshot SNES (MobyGames): mobygames.com/.../snes
- Playthrough completo SNES (NintendoComplete): youtube.com/watch?v=u41lm1M3PpU

Cosa NON è richiesto

Per evitare ansie inutili, chiarisco cosa **non** dovete fare:

- non dovete scrivere codice C#;
- non dovete modellare in Maya (potete, se volete, ma non è valutato);
- non dovete implementare multiplayer (tema del prossimo anno);
- non dovete creare un'IA per avversari (è un single-player time-attack);
- non serve una storia o narrativa elaborata.

Criteri di valutazione

La valutazione segue la griglia ufficiale AANT, articolata su quattro voci più un bonus. **Ogni voce è valutata in trentesimi (su 10)**, e il bonus può aggiungersi sopra. Per ognuna indico qui sotto cosa guardo concretamente in questo esonero.

Voce di griglia	Cosa valuto in questo esonero
Capacità di elaborare i concetti appresi e di prospettare soluzioni creative (<i>Funzionalità implementate nel progetto</i>)	Originalità della meccanica scelta, qualità del level design, funzionalità aggiuntive non strettamente richieste, soluzioni creative nell'uso del toolkit
Capacità di partecipare alle attività proposte (<i>Gestione del progetto in remoto, applicazione della teoria</i>)	Uso corretto e costante di Git (commit chiari, frequenti, leggibili), partecipazione alle review intermedie, applicazione di quanto visto a lezione
Capacità di realizzare esecutivamente un progetto (<i>Pulizia generale del progetto, build, polishing generale</i>)	Pulizia della scena e della gerarchia, naming coerente, build Windows funzionante senza errori, polish complessivo (audio, VFX, illuminazione, UI, feedback, <i>game feel</i>)
Comunicazione del progetto	Qualità e precisione della presentazione finale (PDF/Canva): chiarezza, sintesi, capacità di raccontare scelte di design, demo live ben gestita
Bonus	Funzionalità o cure particolari fuori dall'ordinario, sorprese piacevoli, ricerca personale documentata

In pratica: il voto base si costruisce sulle prime quattro voci (ciascuna /10), e il bonus permette di superare la base ottenuta. La voce *"Realizzare esecutivamente"* — pulizia, build, polish — è quella che misura la differenza tra un progetto "che si vede che è stato finito bene" e uno "lasciato a metà": è un ottimo termometro dello stato di salute generale del lavoro.

Calendario e milestone

Il brief vi viene consegnato su Drive **domenica 10 maggio**. Da lì in poi avete tre giovedì di lezione e una settimana finale di polish prima della consegna.

Data	Cosa succede	Cosa dovete avere
Dom 10 maggio	Brief e starter package distribuiti su Drive	Lettura del brief, primo sguardo al toolkit

Data	Cosa succede	Cosa dovete avere
Gio 14 maggio	Lezione 1 — Kickoff: walkthrough completo del toolkit, esempi di combinazioni, domande aperte	Brief letto, idea di meccanica abbozzata
Gio 21 maggio	Lezione 2 — Mid review: ognuno mostra la propria meccanica, feedback di game design	Livello giocabile end-to-end, anche brutto, repository Git con commit regolari
Gio 28 maggio	Lezione 3 — Polish review: ultimi bug, tips di feel, prep alla presentazione	Livello completo, in fase di rifinitura, build Windows già provata
Gio 4 giugno	Consegna e presentazione finale	Repository Git aggiornato, build Windows su Drive, presentazione PDF/Canva

Modalità di consegna

Stesso schema di consegna del primo esonero. Entro le ore 9:00 del **4 giugno** dovete avere pronti tre artefatti:

- **Progetto Unity 6.0.x su Git** — repository pubblico o privato condiviso col docente. Voglio vedere uno storico di commit regolari (non un singolo commit "consegna finale" la sera prima). Includete un `README.md` con nome, cognome, titolo del gioco, descrizione della meccanica in 2-3 righe, comandi base.
- **Build Windows su Google Drive** — eseguibile pronto all'uso in una cartella nominata `[Cognome]_[NomeGioco]_Build/`. La build deve aprirsi e funzionare al primo lancio, senza dipendenze esterne.
- **Presentazione in PDF o Canva** — stesso formato del primo esonero. Sintetica, visiva, con sezioni dedicate al concept della meccanica, alle scelte di design, agli screenshot del gameplay e a un breve "cosa rifarei / cosa aggiungerei".

In aula, il 4 giugno, ognuno presenta il proprio gioco in **4-5 minuti**: una breve dimostrazione live dalla build, un passaggio sulla presentazione, e Q&A;

Note tecniche

- **Versione Unity**: 6.0.x (la stessa che usiamo in aula). Non aggiornate a versioni successive in corso d'opera.
- **Repository Git**: ricordatevi `.gitignore` corretto per Unity (escludere `Library/`, `Temp/`, `obj/`, `Logs/`, `UserSettings/`). Commit frequenti con messaggi sensati: parte della valutazione passa di qui.
- **Build**: target Windows. Provate la build *prima* dell'ultimo giorno: non è raro che funzioni in editor e si rompa in build.
- **Asset Store**: non aggiungete pacchetti pesanti senza confronto con me. Alcuni rallentano il progetto e generano conflitti con il toolkit fornito.
- **Backup**: Git non basta come unico backup. Tenete una copia anche su Drive o chiavetta. Perdere il lavoro tre giorni prima della consegna è un classico, evitiamolo.

Una parola finale

Questo esonero è progettato per essere **fattibile** se ci lavorate con costanza, e **interessante** se ci mettete idee. Il toolkit toglie l'ostacolo tecnico, ma non sostituisce il pensiero progettuale: la differenza tra un livello mediocre e uno che funziona davvero la fate voi, con le scelte di design, il ritmo, il polish.

Se vi bloccate — su Unity, su un'idea, su qualsiasi cosa — scrivetemi prima della scadenza, non il giorno prima. Buon lavoro.